Machine Learning
Semester 2026-III
Workshop No. 5 — Generative Adversarial Networks

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the fifth workshop of the *Machine Learning* course project!

Your team has explored supervised learning, reinforcement learning, and unsupervised approaches (autoencoders). This workshop introduces **adversarial training**, a fundamentally different learning paradigm. Generative Adversarial Networks (GANs) pit two neural networks — a generator and a discriminator — against each other in a minimax game, enabling the generation of synthetic data.

Generative models address practical challenges including **data scarcity** and **data augmentation**. Well-trained generative models can produce realistic synthetic examples that complement real training data and potentially improve downstream task performance.

The central questions for this workshop are: **Can you develop a GAN capable of generating realistic synthetic data from your domain? How can generated data potentially augment and improve your overall system?**

General Workshop Definition: Generative adversarial networks involve training a generator network G and a discriminator network D jointly through an adversarial objective. G maps a noise vector to synthetic data samples; D tries to distinguish real data from generated data. At convergence, G should produce data that is statistically indistinguishable from real training data. In this workshop, teams implement generative models, evaluate generation quality, analyze failure modes, and explore how generated data can be used for data augmentation or other downstream applications.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **`Vanilla GAN` Implementation:** Train a basic `GAN` on your dataset; generate synthetic data and visually evaluate quality.
- **Stable Training (`WGAN-GP`):** Implement Wasserstein GAN with gradient penalty as a more stable alternative; compare convergence behavior and generation quality against vanilla `GAN`.
- **Generation Quality Evaluation:** Implement quantitative metrics appropriate to your data type (Inception Score, Fréchet distance, or domain-specific metrics); perform qualitative assessment.
- **Failure Mode Analysis:** Diagnose and document training pathologies such as mode collapse, vanishing gradients, or training instability.
- **Data Augmentation Experiment:** Generate synthetic data and retrain the Workshop 2 model on augmented (real + synthetic) data. Compare performance against the baseline trained on real data only.
- **Optional: Conditional Generation:** Implement conditional `GAN` (`cGAN`) to generate class-conditional or labeled synthetic data if your domain has discrete labels.

Theoretical Background and References

1. **Foundational `GAN` Papers:**
   - Goodfellow et al. (2014), "*Generative Adversarial Nets*", *NeurIPS 2014* — the original `GAN` paper. Defines the minimax objective $\min_G \max_D V(G, D) = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$. **Required reading.**
   - Radford et al. (2015), "*Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks*", *ICLR 2016* — introduces `DCGAN`, which uses convolutional layers and establishes architectural guidelines: strided convolutions, batch normalization, `LeakyReLU` in discriminator, `ReLU` in generator.
   - Arjovsky et al. (2017), "*Wasserstein GAN*", *ICML 2017* — proposes Wasserstein distance as a more stable training objective. Eliminates mode collapse and provides meaningful loss curves.
   - Gulrajani et al. (2017), "*Improved Training of Wasserstein GANs*", *NeurIPS 2017* — introduces gradient penalty (`WGAN-GP`), the dominant stable GAN training approach.
2. **`GAN`s for Synthetic Data Generation:**
   - Frid-Adar et al. (2018), "*GAN-Based Synthetic Medical Image Augmentation for Increased `CNN` Performance in Liver Lesion Classification*", *Neurocomputing* — directly demonstrates the data augmentation pipeline: train `GAN`, generate synthetic data, augment training set, measure downstream classifier improvement.

- Xu et al. (2019), "*Modeling Tabular Data using Generative Adversarial Networks*", *arXiv:1907.06315* — `TVAE`: a `GAN` designed for synthetic tabular data generation. Relevant if your domain has structured/tabular data.
- Esteban et al. (2016), "*Real-valued (Medical) Time Series Generation with Recurrent Conditional GANs*", *arXiv:1706.02633* — `RC-GAN` for time series data generation.

3. **Generation Quality Metrics:**

   - **Inception Score (`IS`):** Salimans et al. (2016), "*Improved Techniques for Training GANs*", *NeurIPS 2016*. Measures both quality (generated data should be classifiable) and diversity (coverage across classes).
   - **Fréchet Distance:** Heusel et al. (2017), "*GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*", *NeurIPS 2017*. Measures distance between feature distributions of real and generated data.
   - **Domain-Specific Metrics:** For your specific domain, design metrics that assess whether generated data captures important characteristics. Examples: class balance, feature correlation preservation, diversity of generated samples.
   - **Visual Inspection:** Qualitative assessment by humans remains important. Generate sample data and assess visually or through domain expert evaluation.

4. `GAN` **Training Stability:**

   - Arovsky & Bottou (2017), "*Towards Principled Methods for Training Generative Adversarial Networks*", *ICLR 2017* — theoretical analysis of `GAN` training dynamics. Identifies mode collapse and vanishing gradient problems.
   - Isola et al. (2016), "*Image-to-Image Translation with Conditional Adversarial Networks*", *CVPR 2016* — `pix2pix`, demonstrates conditional GANs for paired data. Applicable if your domain has structured input-output relationships.

Required Artifacts and Methodology

1. **Vanilla `GAN` Architecture and Training:**

   - *Generator G:* Design appropriate to your data type. For image-like data: start with projection of noise to spatial dimensions, then use transpose convolutions with upsampling. For tabular data: use fully-connected layers. Output activation should match data (e.g., `Tanh` for normalized data in $[-1, 1]$).
   - *Discriminator D:* For images: convolutional layers with downsampling. For tabular: fully-connected layers. Final layer: binary classifier (real vs. fake) with `Sigmoid` output.
   - *Loss:* Binary cross-entropy (`BCE`). Train discriminator on real and fake batches, then train generator.

- Use separate `Adam` optimizers ($\text{lr} = 2 \times 10^{-4}$) for G and D. Apply label smoothing (real labels = 0.9) to stabilize training.
- Train for 100+ epochs. Log discriminator accuracy on real/fake and generator loss per epoch. Save generated samples every 10 epochs.

2. **`WGAN-GP` Architecture and Training:**

- Use the same generator architecture as vanilla `GAN`.
- *Critic* (discriminator without final sigmoid): same architecture but outputs real-valued scores (not probabilities).
- *Gradient Penalty:* For each training step, interpolate between real and fake data: $\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$ where $\epsilon \sim \mathcal{U}[0, 1]$. Compute $GP = \lambda \cdot \mathbb{E}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$ with $\lambda = 10$.
- *Training schedule:* Train critic 5 steps for every 1 generator step. The critic loss (Wasserstein distance) should decrease monotonically, indicating stable training.
- Train for 100+ epochs with the same logging as vanilla `GAN`. Compare learning stability.

3. **Generation Quality Evaluation:**

- Generate 1,000–10,000 synthetic samples from both vanilla `GAN` and `WGAN-GP`.
- Compute quantitative metrics: Inception Score or appropriate domain-specific metrics for both models.
- Produce sample grids: show 4x4 or 8x8 grids of real vs. generated samples side by side. Caption to identify source.
- Document failure modes observed during training: mode collapse (all generated samples similar), vanishing gradients, training instability, or artifact patterns. For each issue, describe the symptom and what mitigation was attempted.
- Compare: which model (vanilla or `WGAN-GP`) achieved better quality? More stable training?

4. **Data Augmentation Experiment:**

- Generate 1,000–5,000 synthetic training examples using your trained `GAN`.
- Design three training conditions: (a) real data only (baseline from Workshop 2), (b) real + synthetic from vanilla `GAN`, (c) real + synthetic from `WGAN-GP`.
- Retrain the Workshop 2 model (same architecture, hyperparameters) on each dataset. Ensure fair comparison: same validation/test split, same training budget (epochs or learning steps).
- Report downstream performance metrics for each condition. Does synthetic augmentation improve the model? Does one `GAN` produce better augmentation than the other?
- Analyze: why might synthetic data help or hurt performance? What characteristics of the generated data matter most?

5. **Optional: Conditional GAN:**
    - If your data has discrete labels, implement cGAN: concatenate class labels to generator input and discriminator input.
    - Train separately to generate data for each class. Evaluate whether class-conditional generation produces better labeled synthetic data than unconditional GANs.

Deliverables and Documentation

1. **GAN Report (PDF):**
    - Motivation: why generative modeling for your domain?
    - Vanilla GAN and WGAN-GP architecture descriptions
    - Training curves (generator/discriminator losses, training stability comparison)
    - Generated sample grids and quality metrics
    - Failure mode analysis and mitigation attempts
    - Data augmentation experiment results: performance comparison across three conditions
    - Analysis: did synthetic data help? What factors determined success or failure?
2. **Code:**
    - src/gan.py — Generator, Discriminator, Critic implementations
    - src/train_gan.py — training loop for vanilla GAN and WGAN-GP
    - src/evaluate_gan.py — generation quality evaluation and metrics
    - src/generate_samples.py — sample generation and visualization
    - src/augment_and_retrain.py — data augmentation experiment
    - checkpoints/gan_vanilla_best.pt — best vanilla generator
    - checkpoints/gan_wgan_best.pt — best WGAN-GP generator
    - runs/workshop5/ — training logs and generated samples

Submission Requirements

- The `PDF` report must follow the naming convention: `groupN_workshop5.pdf`
- Submit through the designated course platform by the specified deadline
- Include sample generated images/data in the repository as visual evidence of model quality
- Quantitative metrics must be reported for both architectures (vanilla `GAN` and `WGAN-GP`)
- Results of the data augmentation experiment must be clearly documented with performance comparisons

Grading Note: 40% of the workshop grade will be determined by an in-class quiz administered during the course. This quiz will assess your team's understanding of the adversarial training objective, generator-discriminator dynamics, Wasserstein distance, and gradient penalty. The quiz emphasizes mastery of concepts and the ability to explain GAN training strategies and generation quality.

Important Guidelines and Considerations

- **Mode Collapse is Common:** If all generated samples look nearly identical or cover only a subset of the data distribution, mode collapse has occurred. Diagnose by computing diversity metrics on generated samples. Switch to `WGAN-GP`, reduce generator learning rate, or apply one-sided label smoothing.
- **Start at Low Resolution:** For image-like data, train first at 64×64 resolution until convergence, then optionally scale to higher resolution. Higher resolutions require much more compute and careful tuning.
- **Balance Generator and Discriminator:** If the discriminator wins too easily (100% accuracy on real, 0% on fake within 5 epochs), the generator cannot learn. Apply label smoothing, add noise to discriminator inputs, or reduce discriminator learning rate.
- **Evaluate Beyond Loss Curves:** Generator and discriminator losses alone do not indicate generation quality. Use visual inspection and quantitative metrics. A high or low loss value can be misleading.
- **Synthetic Data Quality Matters:** Poor synthetic data (blurry, mode-collapsed, or unrepresentative) may hurt downstream model performance. Validate generation quality before using for augmentation.

- **Cumulative Results Table:** Add `GAN` results to your comparison table. Track performance across all methods: W1 baselines, W2 supervised, W3 RL (if applicable), W4 unsupervised autoencoders, W5 generative models.

Good luck, and embrace the challenge and creativity of adversarial training — the moment a `GAN` produces its first convincing synthetic examples from pure noise is one of the most visually striking and satisfying results in all of machine learning!